

CSCE 620 COMPUTATIONAL GEOMETRY · MODULE 06 · LECTURE NOTES

Polygon Boolean Operations

Union, intersection, difference — and the hardest robustness problem in the course.

MODULE OUTCOMES

- Implement convex and general polygon clipping.
- Explain the Greiner–Hormann approach and its failure cases.
- Compute winding numbers and apply fill rules.
- Explain why boolean operations are the worst robustness case.
- Apply snap rounding and say what it costs.

1 Clipping against a convex region

```
for each edge of the CONVEX clip region:
    output = []
    for each edge (prev -> curr) of the subject polygon:
        if curr is INSIDE:
            if prev is OUTSIDE: output += intersect(prev, curr)
            output += curr
        else:
            if prev is INSIDE: output += intersect(prev, curr)
    subject = output
```

$O(n*k)$. THE CLIP REGION MUST BE CONVEX -- the half-plane decomposition requires it and there is no way around that.

The known artifact: if the *subject* polygon is concave, the result may contain degenerate connecting edges — a single output ring that runs out to the boundary and back along the same line, joining what should be two separate pieces. **It renders correctly under most fill rules and is not a simple polygon**, so anything downstream that assumes simplicity (triangulation, offsetting, area computation by the shoelace formula) will misbehave. This is the artifact people encounter and cannot explain.

CONVEX CLIPPING COVERS MORE CASES THAN YOU EXPECT

Viewport and scissor clipping — a rectangle, which is convex. This is the single most common clipping operation in graphics.

Frustum clipping — six half-spaces, which is precisely this algorithm in three dimensions, and is what the hardware does between the vertex and fragment stages.

Shadow volume clipping, portal clipping, and splitting a polygon against a BSP plane (Module 07) — all half-space operations.

So reach for Sutherland–Hodgman first, and escalate to a general boolean only when the clip region genuinely is not convex. **The difference in cost is large and the difference in robustness is larger** — convex clipping constructs intersections too, but only against a known half-plane, which is a far better conditioned computation.

2 The general case

GREINER–HORMANN: WALK THE INTERSECTED BOUNDARIES

Find every intersection between the two boundaries and insert each as a new vertex into both polygons' circular linked lists, with cross-links joining the two copies of each intersection.

Classify each intersection as an entry or an exit with respect to the other polygon — determined by whether the boundary is crossing into or out of the other region at that point.

Then walk. Traverse one boundary until reaching an intersection, jump through the cross-link to the other polygon, and continue — switching polygons at every crossing. Each closed walk produces one output ring; repeat until all intersections are visited.

The traversal rule selects the operation. Union, intersection, and difference differ only in which direction you proceed after each jump and which intersections you start from. **One algorithm, three results**, which is the appeal of the formulation.

Degenerate case	What happens
A vertex of one polygon lies exactly on an edge of the other.	The entry/exit classification is undefined — the boundary touches without crossing, so it is neither entering nor exiting. The walk then takes a wrong turn and produces a malformed ring.
Collinear overlapping edges.	There are infinitely many intersection points, so the intersection-insertion step has no well-defined output at all.
The polygons share a vertex.	A degenerate intersection that a general segment-intersection routine may not report, so the walk never switches where it should.
The result touches itself at a single point.	The output topology is genuinely ambiguous — one ring pinched at a point, or two rings meeting? Both are defensible and downstream code cares.

Degenerate case	What happens
Self-intersecting input.	Undefined behaviour; the input must be resolved into simple rings first, which is itself a boolean operation.

The original 1998 Greiner–Hormann paper handles none of these and says so. **Foster, Overfelt and Hormann's 2019 extension handles most of them**, and is the version to implement. A great deal of broken polygon-clipping code in the world is a faithful implementation of the 1998 paper applied to degenerate input.

3 Winding numbers

$w(p)$ = signed number of boundary crossings of any ray from p

Counting counter-clockwise crossings as +1 and clockwise as -1. The value is independent of the ray chosen, which is what makes it well defined. **The non-zero rule** declares p inside when $w(p) \neq 0$ — used by most CAD systems and most renderers, and it requires holes to be wound opposite to their enclosing boundary. **The even-odd rule** declares p inside when $w(p)$ is odd — the PostScript and SVG default, under which a shape that overlaps itself develops a hole. **Mismatched fill rules between an authoring tool and a renderer explain a large fraction of real-world vector rendering bugs.**

BOOLEANS AS WINDING ARITHMETIC

Compute the arrangement of both polygons' edges together, then for each face of the arrangement compute two winding numbers: one with respect to A's edges and one with respect to B's.

Each operation is then a predicate on that pair. Union keeps faces where $w_A \neq 0$ or $w_B \neq 0$; intersection where both are nonzero; difference where $w_A \neq 0$ and $w_B = 0$; symmetric difference where exactly one is.

So the algorithm is one arrangement computation plus a per-face test, rather than three distinct traversal rules with three sets of degenerate cases.

And it extends to many polygons at once, and to self-intersecting input, neither of which the boundary-walk formulation handles. **This is what production libraries actually do** — Clipper, CGAL's Nef polyhedra, and the boolean operations in most CAD kernels are all arrangement-plus-predicate, not boundary walks.

4 Why booleans are the worst robustness case

BOOLEANS CONSTRUCT NEW POINTS, AND THAT IS THE PROBLEM

Every algorithm in this course so far only *classified* existing input points. Convex hull decides which input vertices are extreme; triangulation decides which input vertices to join; the sweep decides orderings. Exact predicates (Module 02) make all of that exact, and nothing more is needed.

Boolean operations *construct* new points — the intersections — and **the intersection of two segments with representable endpoints is in general not representable.** It is a rational number whose denominator is a determinant, and rounding it to the nearest double moves it off both lines.

So the output of one boolean is approximate input to the next. A chain of operations — which is exactly what a CSG modelling session or a mesh-processing pipeline is — accumulates error, and the accumulated error eventually produces self-intersections, slivers, and non-manifold output.

Exact predicates do not solve this. They make every classification correct; they cannot make an unrepresentable coordinate representable. **This predicate/construction distinction is the real lesson of the module**, and it is why CGAL's kernels are parameterised separately on predicate exactness and construction exactness — the two are genuinely different properties with genuinely different costs.

Strategy	Cost	Where it is used
Exact rational coordinates.	Denominators grow with each operation, without bound. After a few hundred chained booleans the numbers are enormous and everything slows to a crawl.	CGAL's exact-construction kernels; high-end CAD. Correct, and genuinely expensive.
Snap rounding.	Moves points, and can change topology — see below.	Most robust practical systems. The standard engineering answer.
Integer coordinates throughout.	Requires fixing a grid resolution in advance and scaling all input onto it.	Clipper, and many 2D libraries. If the domain permits it, the problem largely disappears — intersections still need rounding, but the inputs are exact and comparisons are trivially reliable. The most underrated answer on this list.
Interval arithmetic with exact fallback.	Fast path with a certified slow path, like Module 02's filter applied to constructions.	CGAL's lazy kernel. Keeps exactness while usually paying floating point prices.
Tolerance-based vertex merging.	Unsound — Module 01 §3 applies in full.	Nearly every mesh tool. It works most of the time and fails in ways nobody can reproduce.

SNAP ROUNDING, AND ITS HONEST COST

Round every input vertex and every computed intersection to a fixed grid, then re-resolve any *new* intersections that the rounding itself created — because moving points onto a grid can make previously disjoint edges cross.

The output is then exactly representable, so the next operation in a chain starts from clean input and error stops accumulating. That property is what makes long CSG chains viable at all.

But rounding can change topology. A thin sliver can collapse to nothing; two separate components can merge into one; a vertex can cross an edge it was formerly on the other side of. None of these is detectable from the output alone.

So the result is a valid polygon *near* the true answer, which is normally acceptable and must be documented as what the function promises. **Iterated snap rounding and its variants bound the total displacement** of any point, which is the principled version and the one to cite when someone asks how wrong the answer can be.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Greiner & Hormann — Efficient Clipping of Arbitrary Polygons (free)	The 1998 original of §2. Read it, then read the 2019 extension before implementing.

Resource	What it covers for this module
Foster, Overfelt & Hormann — Clipping simple polygons with degenerate intersections (free)	The degeneracy handling the original lacks. This is the version to implement.
Angus Johnson — Clipper2 (free, Boost licence)	The integer-coordinate strategy of §4, shipped. Widely used, readable, and the best evidence for that row of the table.
Hobby — Practical segment intersection with finite precision output; and CGAL's snap rounding package (free)	Snap rounding, stated properly, with the displacement bounds.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement Sutherland–Hodgman and clip a concave subject against a rectangle. **Produce the degenerate connecting edge and draw it.**
- 2 Extend it to 3D frustum clipping against six planes.
- 3 Implement Greiner–Hormann for union, intersection, and difference.
- 4 **Construct all five degenerate cases** from §2's table and report what your implementation does with each.
- 5 Implement the 2019 degeneracy extension and re-run them.
- 6 Implement winding-number computation and both fill rules. Find a polygon where they disagree and render both.
- 7 **Implement booleans as arrangement plus per-face predicate** and compare robustness against the boundary walk.
- 8 **Chain 100 random booleans** with double coordinates and measure how the error and the vertex count grow.
- 9 Repeat with snap rounding and with integer coordinates. Compare.
- 10 Find an input where snap rounding changes the topology, and characterise it.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State Sutherland–Hodgman and its one hard requirement.
- 2 What artifact appears when the subject polygon is concave?
- 3 Name four uses of convex clipping.

- 4 Describe the four steps of Greiner–Hormann.
- 5 Give five degenerate cases that break it.
- 6 Define the winding number and both fill rules.
- 7 Why are booleans harder than everything before them?
- 8 Distinguish predicates from constructions, and say why it matters.
- 9 Give five robustness strategies and the cost of each.